

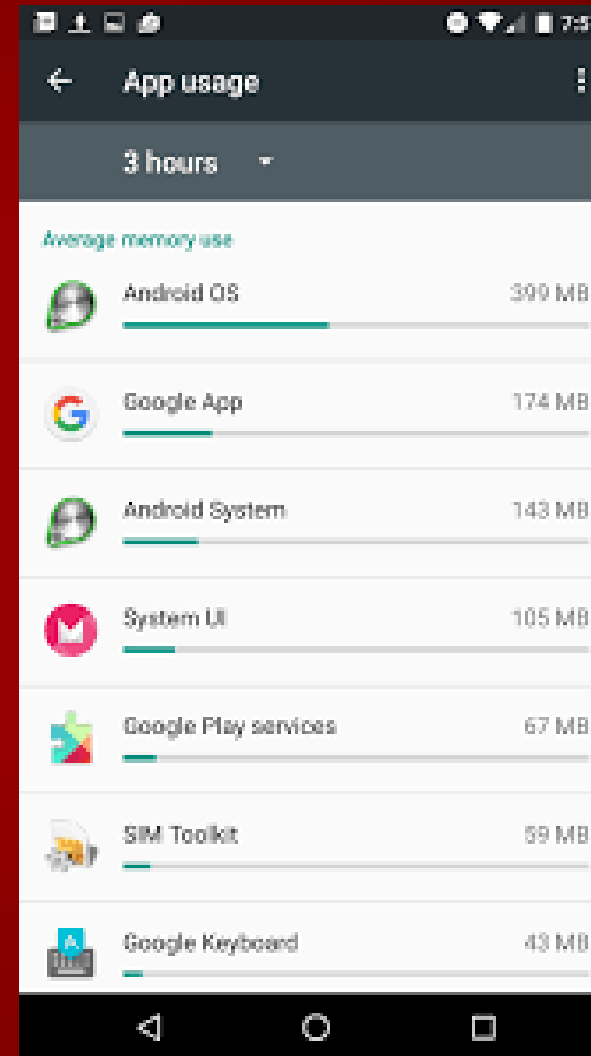
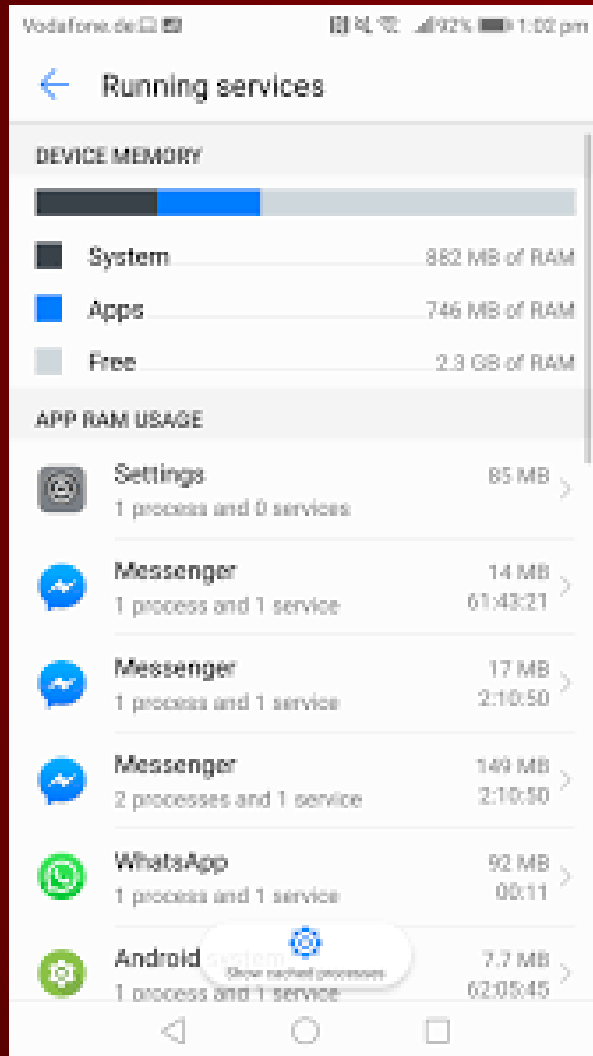
SERVICES

Prof.K.Sathiyamurthy
DCSE,PTU

Services

- ❖ Visible items in the android screen is an Activity
- ❖ Background Tasks – carried by service
 - ❖ Playing Music
 - ❖ Downloading a File
 - ❖ Updating apps
 - ❖ Whatsapp, facebook apps – fetching data from servers in background
- ❖ Service is an android component that allows you to perform tasks in the background without disturbing your current interaction with the activity

Services - Example



Services - Introduction

- ❑ Application component that executes tasks in the background
- ❑ No user Interface
- ❑ By default runs in app's main thread
 - ❑ Does not run in Separate thread
 - ❑ Does not run in separate process

Types of Services

1. Scheduled Service



2. Started Service



3. Bound Service



*Bound Service provides Two way communication service

Scheduled Service

- ❑ Introduced in API 21 (Android 5.0)
 - ❑ Can't be used for below API 21 devices
- ❑ To run or execute a task
 - ❑ Based on certain specific conditions
 - ❑ Not at specific time
- ❑ Replacement of alarm manager – as it was time specific and not condition specific
- ❑ Examples
 - ❑ Sync data only when connected with wifi
 - ❑ Start downloading the file only when the device is plugged in

Started Service

- ❑ Started by android component – Activity, Service, BroadcastReceiver, ContentProvider
 - ❑ `startService()`
- ❑ When started can run in background indefinitely – even when the component that created the service is destroyed
- ❑ Performs single operation
- ❑ By default, does not return anything back to the caller
 - ❑ Use `ResultReceiver`, `BroadcastReceiver` or `BoundService`
- ❑ Call `stopSelf()` or `stopService()` after the task is finished
- ❑ *2026 update: on API 26+ (Android 8.0+), you can't call `startService()` while your app is in the background — use `startForegroundService()` instead (see *Foreground Service, ahead*)*

Bound Service

- ❑ Started by android component(Activity/Provider/Service)
- ❑ Android components that can bind to the service are (Activity/Provider/Service)
- ❑ Broadcast Receiver cannot bind to the service
- ❑ Method-bindService()
- ❑ Exists as long as there is atleast one component bound to it
 - ❑ When all the calling component are destroyed, the service is also destroyed
 - ❑ Service is totally dependent on the calling component
- ❑ Continuously interact with calling component
- ❑ By default has mechanism to return result back to the caller

MyAndroidApplication

A Default Process is created

Main Thread (UI Thread)

MainActivity

MyProvider

MyService

MyReceiver

All components run in same process

Background or worker Thread one

Operations:

- Small Operations
- Light weight operations

Operations:

- Long Operations
- Time taking operations

MyAndroidApplication

A Default Process is created

Main Thread (UI Thread)

MainActivity

MyProvider

MyService

MyReceiver

All components run in same process

Background or worker Thread one

Process 2

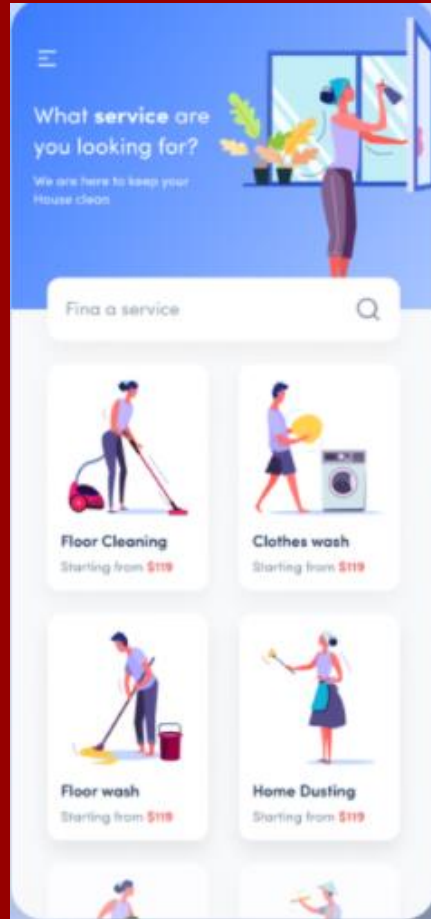
Default Main Thread

MySecondService

worker Thread

Default Process

Main UI Thread



UI
Interaction

Mathematical
Operations

Small
Operations

Button
Click

Small logical
Operations

Main UI Thread

Worker or Background Thread

Long
Operations

File Download

Network
Operations

Image
Loading

Database
Queries

STARTED SERVICE

Started Service

- ❑ Runs on the main thread despite being independent of the calling component
 - ❑ May block UI if executed for long duration
- ❑ Used to perform single task that does not return anything to the calling component
- ❑ Started Service can receive data through an intent, but cannot return data back to the calling component
 - ❑ Can use Bound Service to receive
 - ❑ Implement a receiver to receive data back

Starting a Started Service

- ❑ Use `start service(intent)` to start service
- ❑ If the service is not running, it will be instantiated and started
 - ❑ `OnCreate` (to instantiate service)
 - ❑ `OnStartCommand` (to start service)
- ❑ `StartService` always runs in the main thread (in background)
- ❑ `StartService` is a method of `Context`(super class of activity)
 - ❑ Use it inside the Activity class
 - ❑ Our responsibility to stop service

Steps to create a Started Service

- ❑ Sub class Service – MyStartedService.java
- ❑ Override methods
 - ❑ **onStartCommand**
 - ❑ Should be overridden for started service
 - ❑ Once executed the service will run until either stopself(in the service) or stopService(in the calling component) is called
 - ❑ **onBind**
 - ❑ Always required
 - ❑ For a started service, return null
 - ❑ **onCreate**
 - ❑ Called if the service is not already running and needs to be created
 - ❑ **onDestroy**
 - ❑ Called when the service is no longer being used
- ❑ **Since service is an application component it has to be declared in AndroidManifest.xml**

Stopping a Started Service

- ❑ `stopService(intent)`
 - ❑ A startedService will run until it receives a call to `stopService(intent)`
 - ❑ `stopService` stops a started service no matter how many times `startService( )` was called earlier
 - ❑ If the service is not running calling `stopService` does nothing
- ❑ `stopSelf()`
 - ❑ Called from within the Service class itself

Started Service- onStartCommand return Flags

- ❑ Sometimes Android system may terminate the service(may be due to lack of memory)
- ❑ START_STICKY
 - ❑ Service restarts automatically
 - ❑ Intent Lost(becomes NULL)
- ❑ START_REDELIVER_INTENT
 - ❑ Service restarts automatically
 - ❑ Intent redelivered
- ❑ START_NOT_STICKY
 - ❑ Service not restarted

Started Service- onStartCommand return Flags

- ❑ StartedService works in main thread
 - ❑ Never block the main thread as it will block the UI
 - ❑ System will generate ANR dialog
 - ❑ App will finally crash
- ❑ Solution
 - ❑ Perform the long running task in background thread

Activity (Main Thread)



Started Service
onStartCommand
(Main Thread)



Async Task

onPreExecute
(Main Thread)



doInBackground
(Background Thread)



onPostExecute
(Main Thread)

2026 update: AsyncTask is deprecated since API 30 (Android 11). Prefer Kotlin Coroutines or java.util.concurrent Executors in new code.

FOREGROUND SERVICE

Foreground Service

- ❑ For ongoing, user-visible work — music playback, navigation, an active call, fitness tracking (this deck's own “Playing Music” example needs this)
 - ❑ Must show a persistent notification the whole time it runs
- ❑ Since API 26 (Android 8.0): use `startForegroundService()` instead of `startService()` when the app is in the background
 - ❑ The service must call `startForeground()` within 5 seconds, or the system stops it

Foreground Service

- ❑ Since API 34 (Android 14): must declare a foregroundServiceType in the manifest and hold the matching permission
 - ❑ e.g. FOREGROUND_SERVICE_MEDIA_PLAYBACK, FOREGROUND_SERVICE_LOCATION
- ❑ Google's current guidance: prefer WorkManager for most background work
 - ❑ Reach for a Foreground Service directly only when the task is genuinely ongoing and user-visible

Intent Service

*2026 update: deprecated since Android 11 (API 30) — Google now recommends
WorkManager*

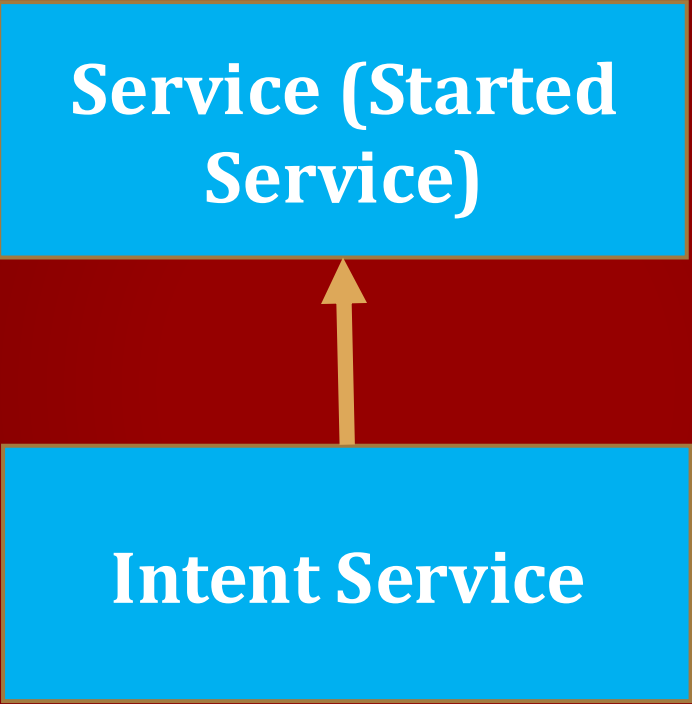
Intent Service

Super class

Service (Started
Service)

Sub class

Intent Service



```
graph BT; A[Intent Service] --> B[Service (Started Service)];
```

Intent Service

- ❑ Runs on a separate thread [Worker Thread]
- ❑ Need to override onHandleIntent()
- ❑ Create a work queue to handle all intents one at a time
 - ❑ Hence handles multi-threading details internally
- ❑ Automatically stops itself when work is complete
 - ❑ No need to call stopService or stopSelf

Intent Service

- ❑ Provides default implementation of onBind method that returns null
 - ❑ It is a variant of startedService
- ❑ Provides default implementation of onStartCommand
 - ❑ onStartCommand->Work Queue (handles multithreading)-> OnHandleIntent

Started Service vs Intent Service

Started Service	Intent Service
Operates in MainThread	Operates in Worker Thread
May Block the UI	Does not block the UI
Used for Shorter duration	Used for longer Process

2026 update: Intent Service is deprecated (API 30) — prefer WorkManager for new code

Communication Flow Between Activity and Service

Activity and Service communication

PROBLEM STATEMENT

- ❑ Sometimes an Activity that starts a service needs data back from the service
 - ❑ Services don't have a native mechanism to return a result through the intent
 - ❑ Sometimes we need the result back to update UI element

Activity and Service communication

1. BroadcastReceiver: For within and cross app communication
(within 2 or more app communications)



- a. Local Broadcast : If within the same application
- b. Normal Broadcast: If between two different apps

2. ResultReceiver: For within app only
(Communication within same application)



Activity and Service communication

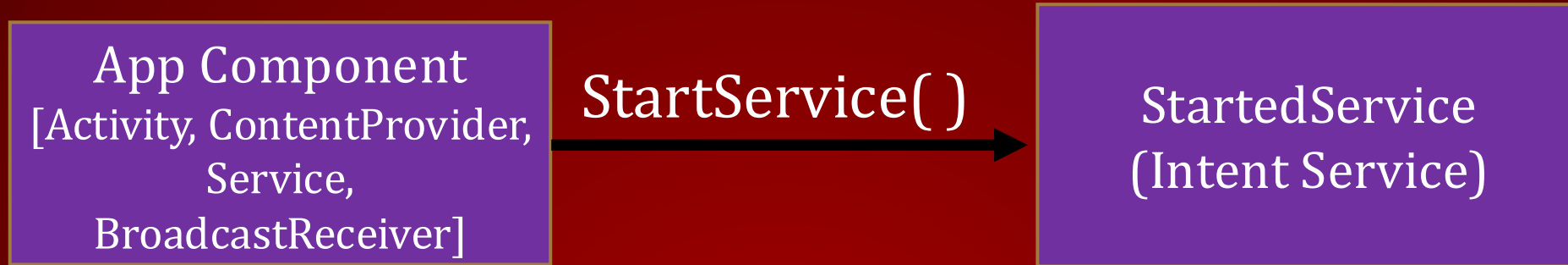
SUMMARY

- ❑ Within Same app
 - ❑ `ResultReceiver` and `BroadcastReceiver`
- ❑ Outside the app
 - ❑ `BroadcastReceiver`

2026 update: `LocalBroadcastManager` (the classic “Local Broadcast” API) is deprecated. For in-app communication, prefer a direct callback, `LiveData/Flow`, or a simple in-process event bus.

BOUND SERVICE

BOUND SERVICE



If the calling component is destroyed -> Started Service continues to run normally
-> until stop service or stopSelf is called
-> stops automatically if it is intentService



Bound Service is totally dependent on calling component

If the calling component is destroyed -> Bound Service is also Destroyed simultaneously

*Broadcast Receiver cannot to used as a calling component

*Bound service has native mechanism to communicate to the calling component

BOUND SERVICE

- ❑ Works in Main Thread
 - ❑ Don't perform long operations
- ❑ Based on client-server relationship principle
 - ❑ Calling component:client
 - ❑ Bound Service : server
 - ❑ Bound service [server] continuously interact with the calling component[client]
- ❑ Any app component can bind to this service
 - ❑ Except broadcast receiver
- ❑ Destroyed if no active component is bound to this service

* Bound Services , you can bind this working component with the activity and then dispatch the results on to foreground(The Ui)

BOUND SERVICE

- ❑ onBind() method returns IBinder
 - ❑ Unlike started service that returns null – it should return the instance of binder object
- ❑ Invoked by bindService()
- ❑ Unbind from this service by unbindService()

Bound Service Code snippet

Calling Component: SecondActivity.java

```
// App Opens
bindService(intent, mConnection, flag);

private ServiceConnection mConnection . . .{
    onServiceConnected(name, IBinder iBinder){
        // We get the instance of
        // 1. Local Binder- myLocalBinder
        // 2. Bound Service- myBoundService
    }
}

// Perform tasks: myBoundService.add();

// App Close
unbindService(mConnection);
```

MyBoundService.java

```
public class MyLocalBinder extends Binder {
    return MyBoundService.this; // Instance
}

Binder myLocalBinder = new MyLocalBinder();

public IBinder onBind(Intent intent) {
    return myLocalBinder; // Instance of Binder
}

public void add() {
    // Add two numbers
}
```

FLAGS

- ☐ BIND_AUTO_CREATE
 - ☐ Automatically creates service as long as components are bound to it
- ☐ BIND_DEBUG_UNBIND
 - ☐ Creates debug information for unmatched bind/unbind calls
- ☐ BIND_NOT_FOREGROUND
 - ☐ The bound service will never be brought to the foreground process level
- ☐ 0 [zero]
 - ☐ None

STARTED SERVICE vs BOUND SERVICE

Started Service/ Intent Service	Bound Service
Just to accomplish task [May be long Task]	For long-standing connection
Invoked by <code>startService()</code>	Invoked by <code>bindService()</code>
<code>onBind()</code> returns null	<code>onBind()</code> returns <code>IBinder</code>
Continue to run even if the calling component is destroyed	If the calling component is destroyed then Bound service too gets destroyed

- ❑ Generally we use `onStart()` and `onStop()` method
 - ❑ To bind or unbind the service
 - ❑ So, the Bound service is only active when the Activity is visible to the user
- ❑ If you want your activity to bind to the service even when it is not visible
 - ❑ i.e. the activity in paused state
 - ❑ Call `bindService()` in `onCreate()`
 - ❑ Call `unbindService()` in `onDestroy()`